

Event-Driven Architecture

Event-Driven Architecture

- Estilo arquitetural distribuído muito popular, altamente escalável e de alto desempenho.
- Baseada no desacoplamento de componentes a partir de eventos, que produzem dados de forma assíncrona e outro componente é responsável por processar estes dados.
- Pode ser usado de forma única ou em conjunto com outras arquiteturas, por exemplo, a Hexagonal, Microkernel ou Serviços / Microserviços.
- A maioria das aplicações segue um padrão do “produtor-consumidor” onde requisições dos clientes são enviadas a um servidor para orquestração, cada requisição pode ser redirecionada para um outro servidor especializado ou uma instância de “consumidor”.

Event-Driven Architecture

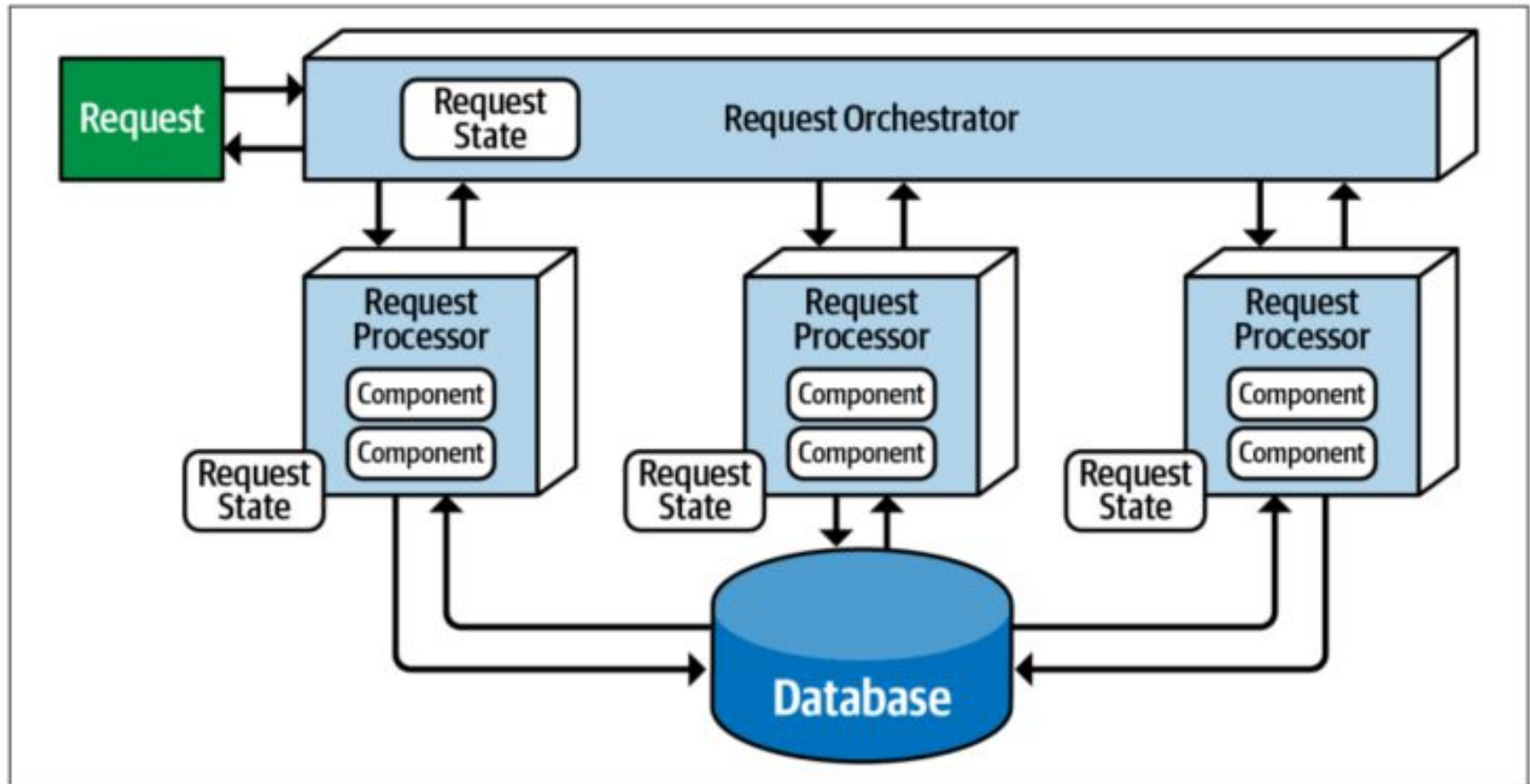


Figure 14-1. Request-based model

Uma pequena pausa para “Event-Driven Architecture + Hexagonal” Exemplo prático

Event-Driven Architecture + Hexagonal

- Vamos modelar um **sistema de gerenciamento de pedidos** virtual
 - Os pedidos podem ser criados, atualizados e cancelados.
 - Integração com um sistema de pagamento externo.
 - Integração com uma plataforma de envio (logística).
 - Persistência de dados em um banco de dados.
 - Eventos disparam ações como envio de notificações por e-mail, SMS e Whatsapp.

Event-Driven Architecture + Hexagonal

- Núcleo Hexagonal (Domínio)
 - Contém toda a lógica de negócio relacionada aos pedidos.
- Portas e Adaptadores
 - Facilitam a comunicação entre o núcleo e o mundo externo.
- Eventos (Event-Driven)
 - Utilizados para integrar módulos internos e sistemas externos de forma desacoplada.

Event-Driven Architecture + Hexagonal

- Componentes Chave:
 - Domínio do Pedido (Hexagonal): Responsável por gerenciar pedidos.
 - Message Broker: Gerencia a publicação e o consumo de eventos.
 - Sistemas Externos: Como sistemas de pagamento e logística.
 - Banco de Dados: Para persistência dos pedidos e eventos.
 - Notificações: Para informar clientes sobre mudanças no status do pedido.

Event-Driven Architecture + Hexagonal

- Núcleo Hexagonal (Domínio)
 - Regras de negócio centralizadas no domínio do pedido.
 - Manipulação de entidades e serviços de negócio.
 - Define portas (interfaces) para comunicação com o exterior.
- Principais Portas:
 - Portas de Entrada (Input Ports)
 - CriarPedidoService
 - AtualizarPedidoService
 - CancelarPedidoService
 - Portas de Saída (Output Ports):
 - PedidoRepositoryInterface
 - Para persistência dos dados.
 - PagamentoGatewayInterface
 - Para comunicação com o sistema de pagamento.
 - LogisticaServiceInterface
 - Para integração com a logística.
 - NotificationServiceInterface

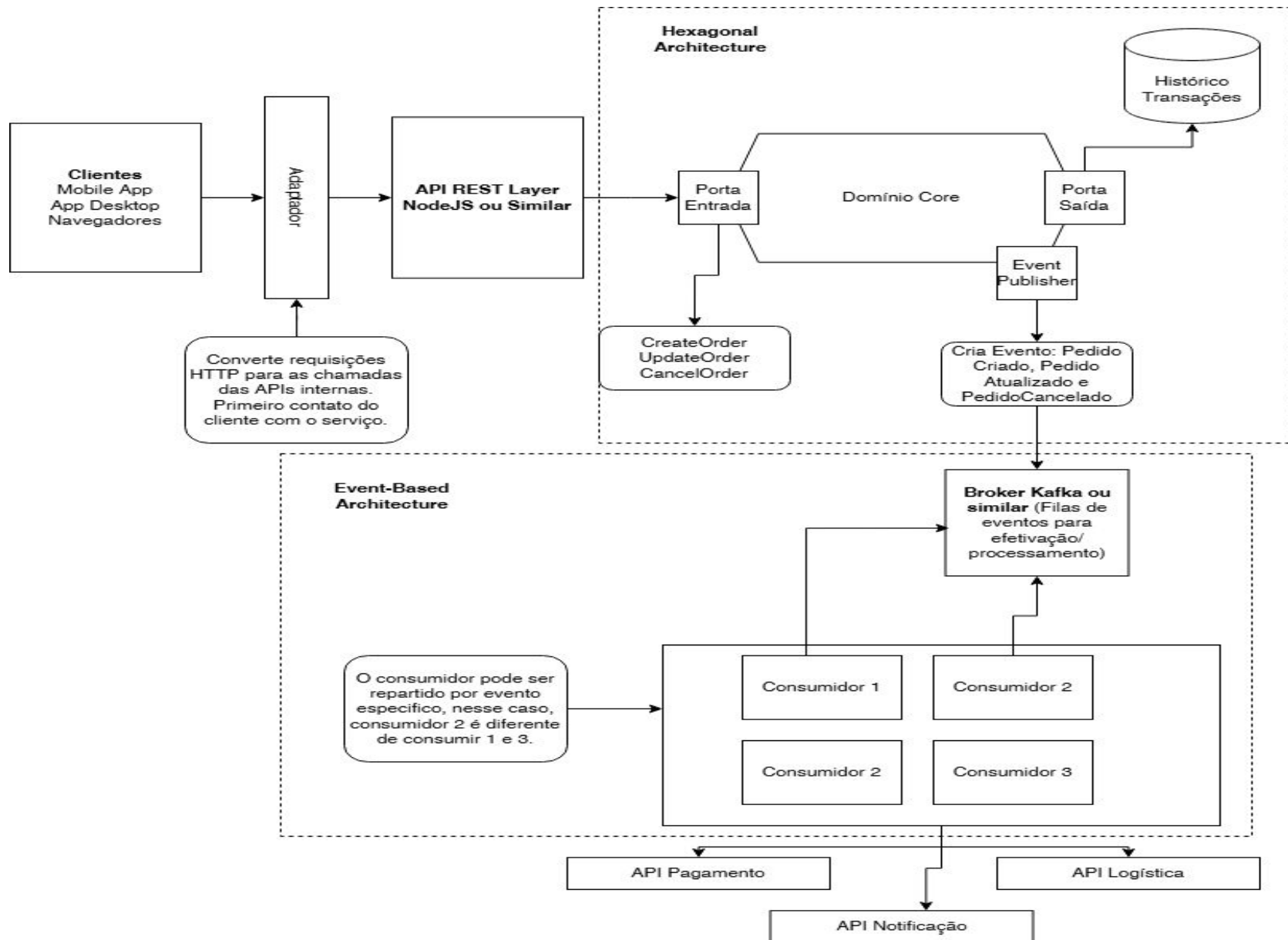
Event-Driven Architecture + Hexagonal

- Event-Driven Architecture
 - Message Broker: RabbitMQ ou Kafka
 - Centraliza a comunicação baseada em eventos entre módulos.
 - Permite que o sistema reaja a mudanças de forma desacoplada.
- Eventos Principais:
 - PedidoCriado: Disparado quando um pedido é criado.
 - PedidoAtualizado: Disparado quando um pedido é atualizado.
 - PedidoCancelado: Disparado quando um pedido é cancelado.
 - PagamentoConfirmado: Disparado quando o pagamento é concluído.

Event-Driven Architecture + Hexagonal

1. Um cliente cria um pedido através da API REST.
2. O núcleo valida e cria o pedido.
3. O evento PedidoCriado é publicado no Message Broker.
4. Os seguintes consumidores reagem ao evento (pode ser assíncrono):
 - a. Serviço de Pagamento: Inicia a cobrança.
 - b. Serviço de Notificação: Envia uma confirmação ao cliente.
 - c. Serviço de Logística: Prepara o processamento do envio.

Event-Driven Architecture + Hexagonal



Event-Driven Architecture + Hexagonal

- Benefícios da Combinação
 - Modularidade
 - A hexagonal organiza o sistema em módulos altamente desacoplados.
 - Escalabilidade
 - A baseada em eventos permite que novos consumidores e emissores de eventos sejam adicionados facilmente.
 - Manutenção
 - Alterar a infraestrutura ou os consumidores não afeta o núcleo.
 - Resiliência
 - Os eventos podem ser processados de forma assíncrona, reduzindo o impacto de falhas.
 - Escalabilidade
 - Mais consumidores podem ser criados para melhorar o fluxo e atender grande quantidade de pedidos.

Voltando para a Event-Driven Architecture somente...

Event-Driven Architecture

- Um modelo baseado em eventos reage a uma determinada situação e aplica uma ação aquele evento.
- Um exemplo é a submissão de uma oferta em um leilão online, onde o cliente faz um registro de oferta, mas essa oferta não é a efetivação da compra.
- O registro da oferta é um evento registrado no sistema que, após a conclusão do leilão serão executadas as ações de escolha do maior lance, efetivação da venda e processamento do pagamento.

Event-Driven Architecture

- A arquitetura baseada em eventos pode ser modelada de duas formas:
 - *Mediator topology*
 - *Broker topology*
- No *mediator* há uma coordenação central onde a requisição é quebrada e enviada para cada componente de forma coordenada, gerindo todo o processo e agregando as respostas.
 - Usada quando há muitos passos que precisam ser coordenados ou sequenciados. Se usamos com a Hexagonal, parece estar mais associada a um *mediator*.
- No *broker* há uma abordagem mais distribuída, onde o *broker* encaminha as requisições para módulos ou servidores que são responsáveis por processar a solicitação.

Event-Driven Architecture

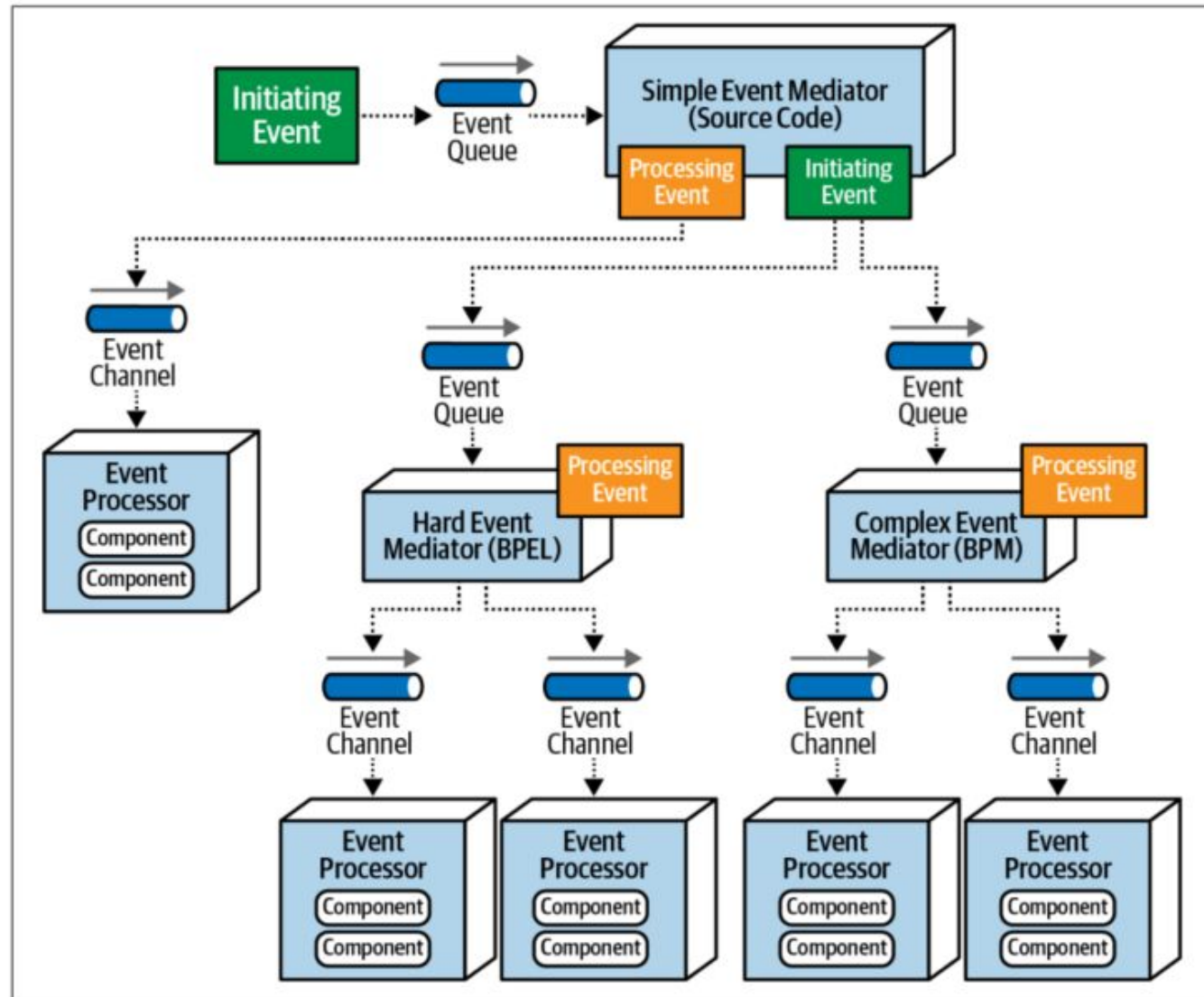


Figure 14-6. Delegating the event to the appropriate type of event mediator

Event-Driven Architecture

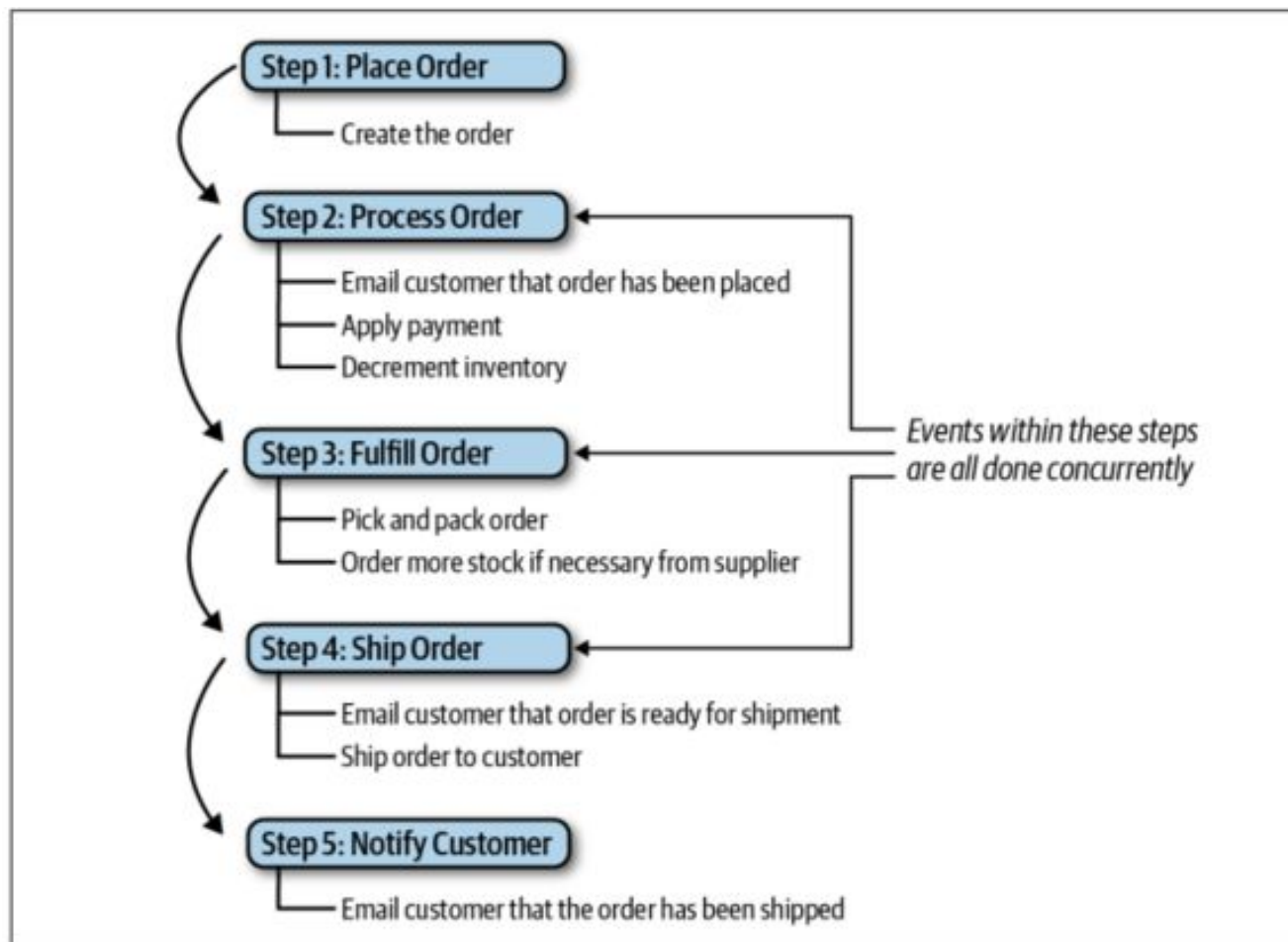
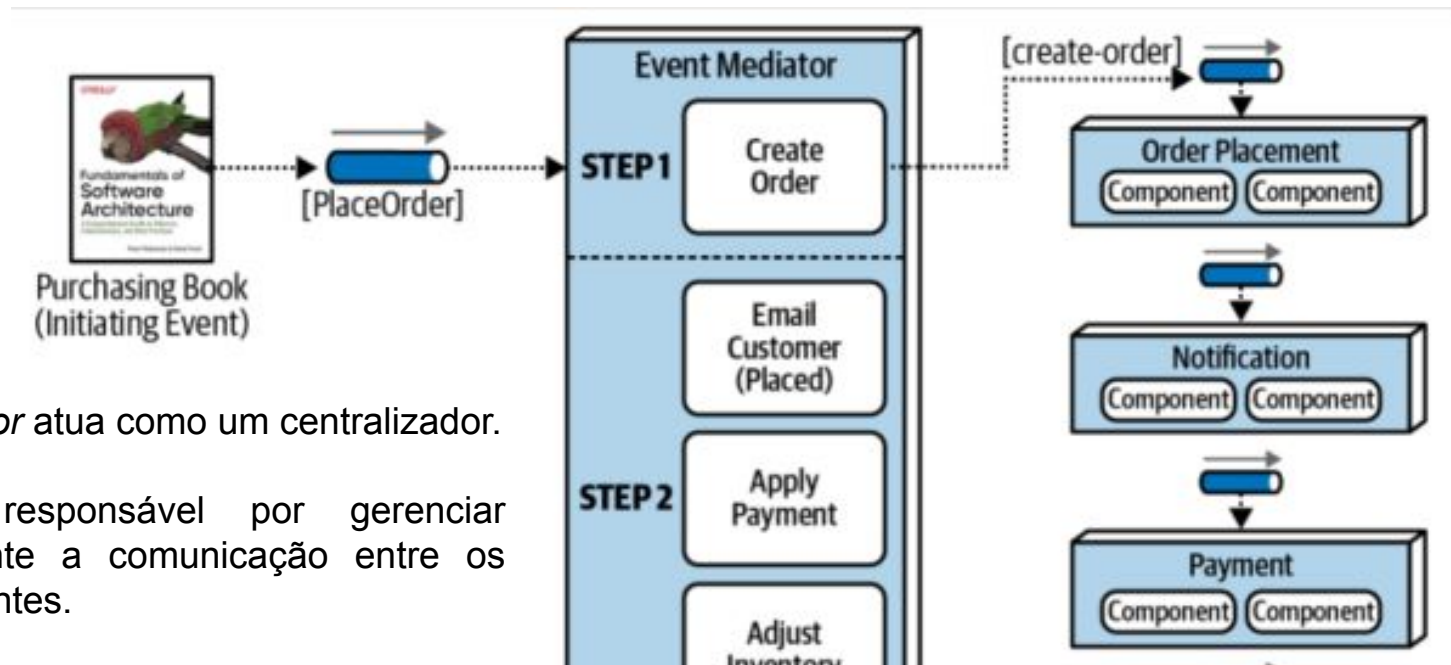


Figure 14-7. Mediator steps for placing an order

Event-Driven Architecture

- Sequência dos eventos e ações ao construir uma abordagem baseada no *mediator*



O *Mediator* atua como um centralizador.

Ele é responsável por gerenciar diretamente a comunicação entre os componentes.

Os emissores enviam eventos para o Mediator, que os direciona explicitamente aos receptores apropriados

Event-Driven Architecture

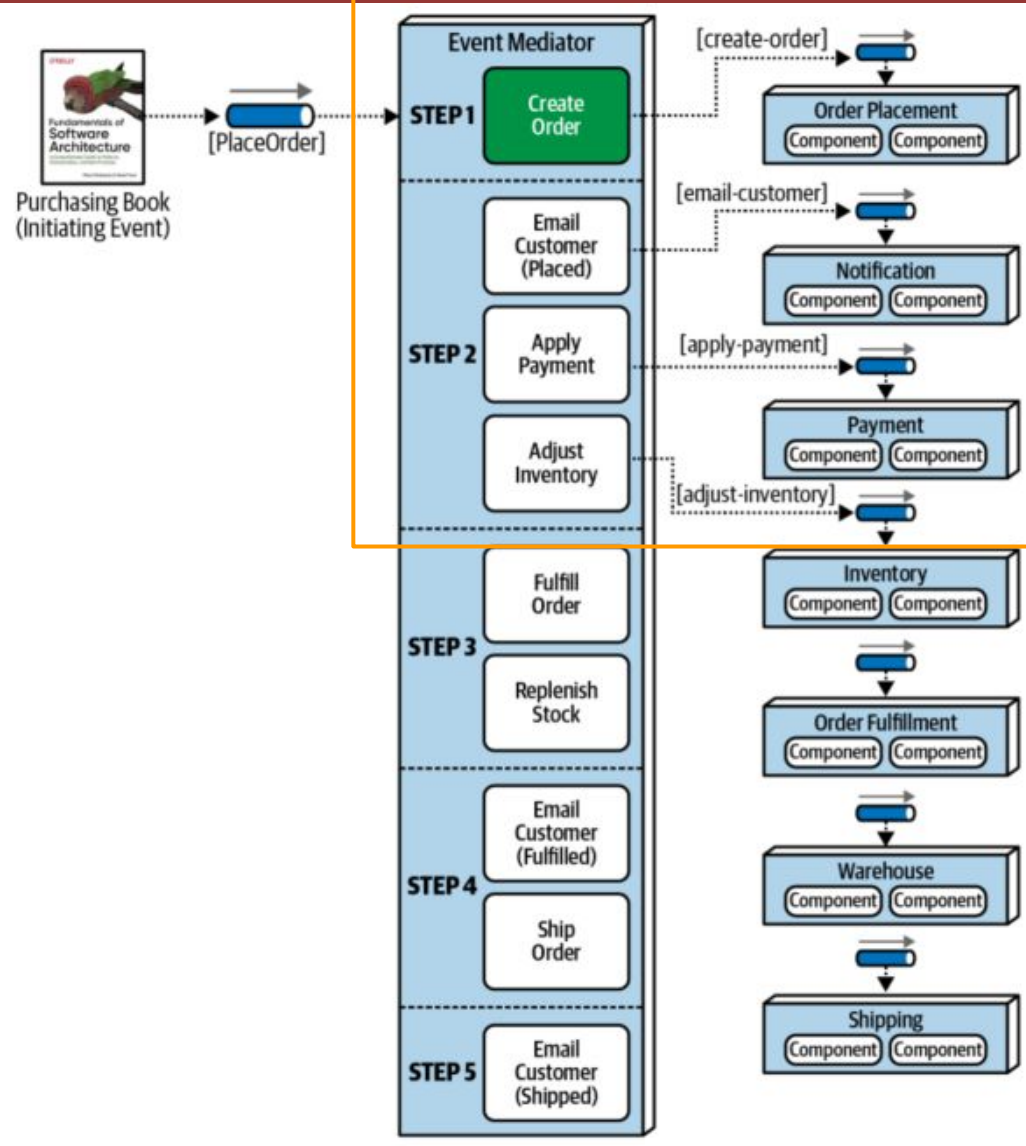
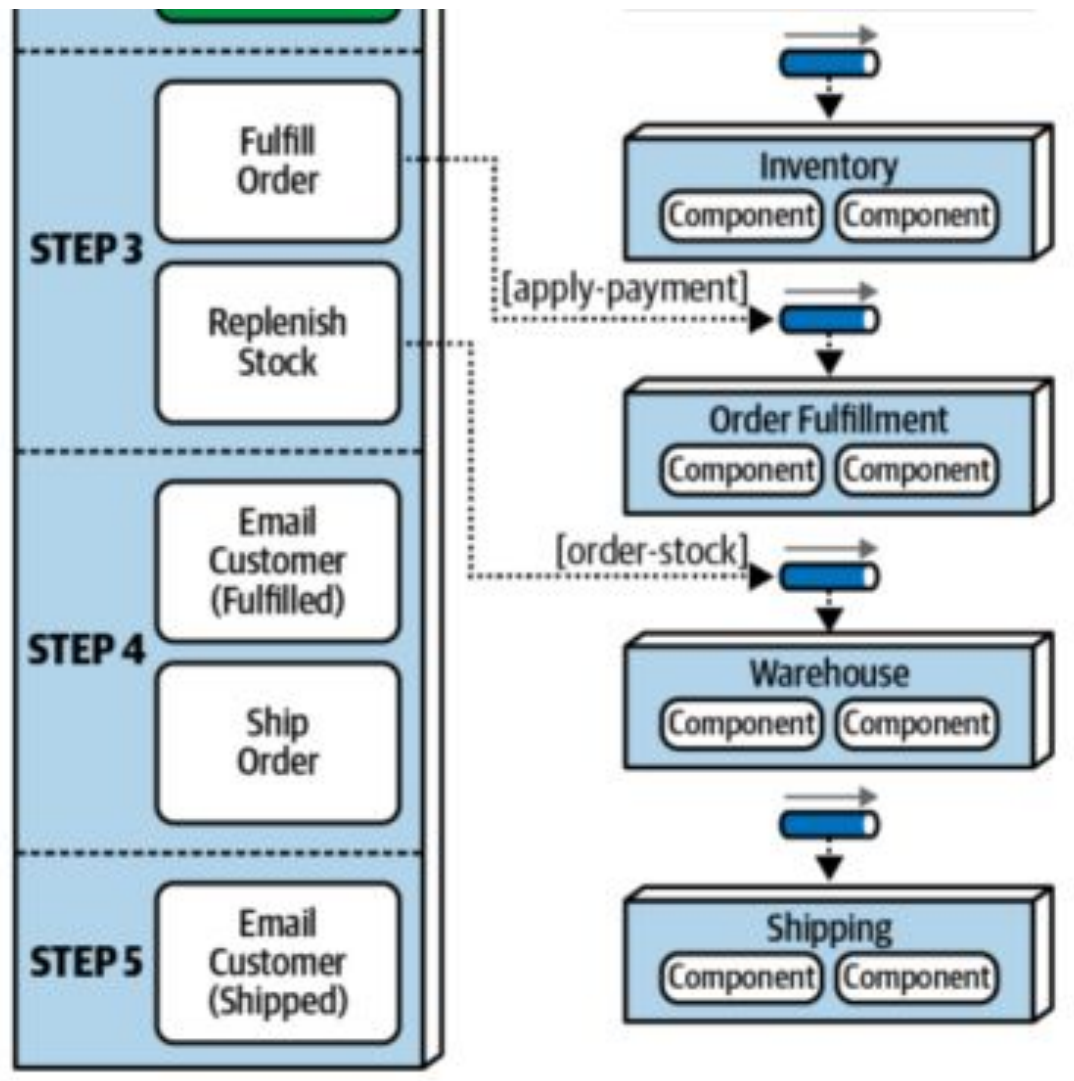
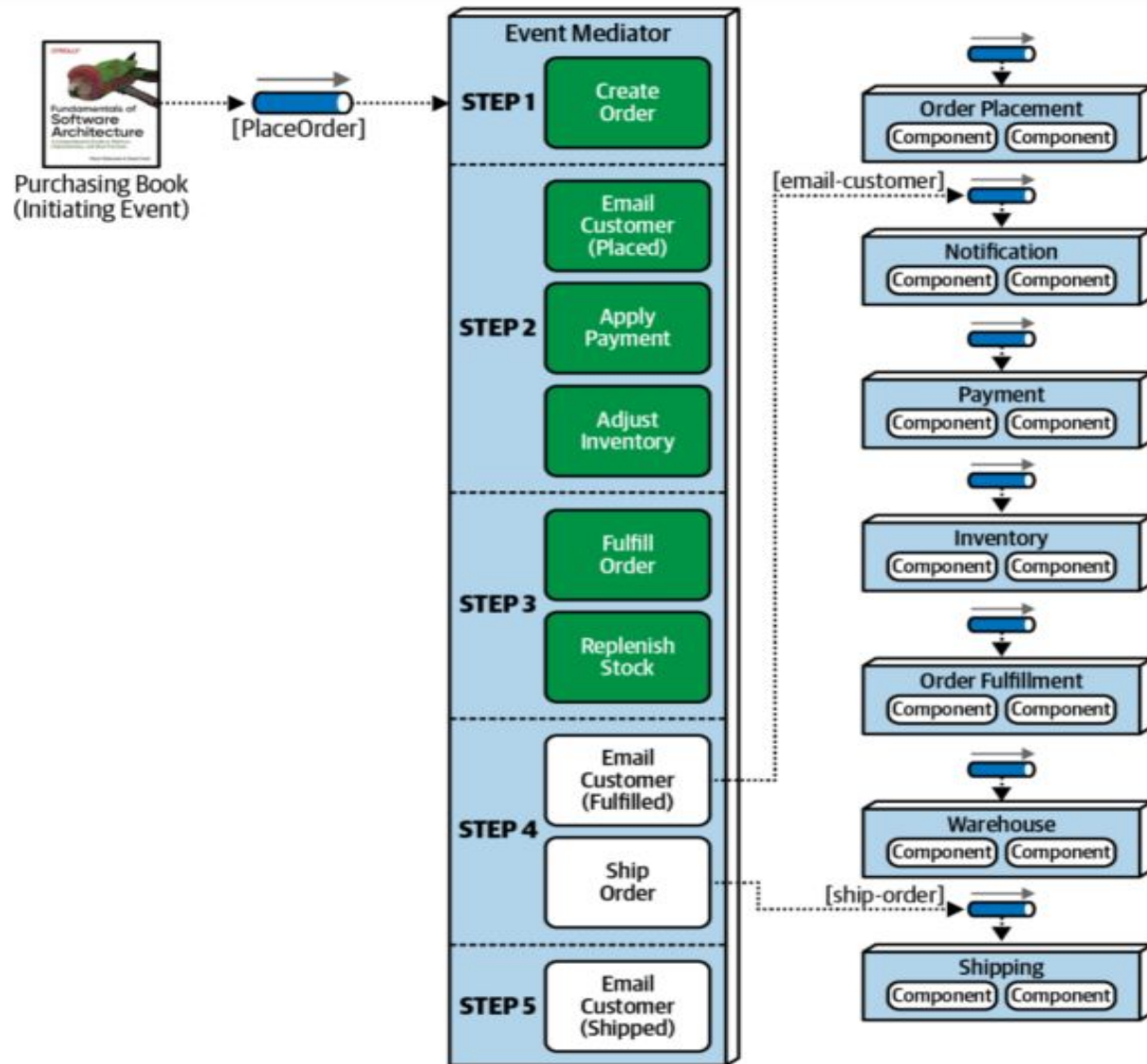


Figure 14-9. Step 2 of the mediator example

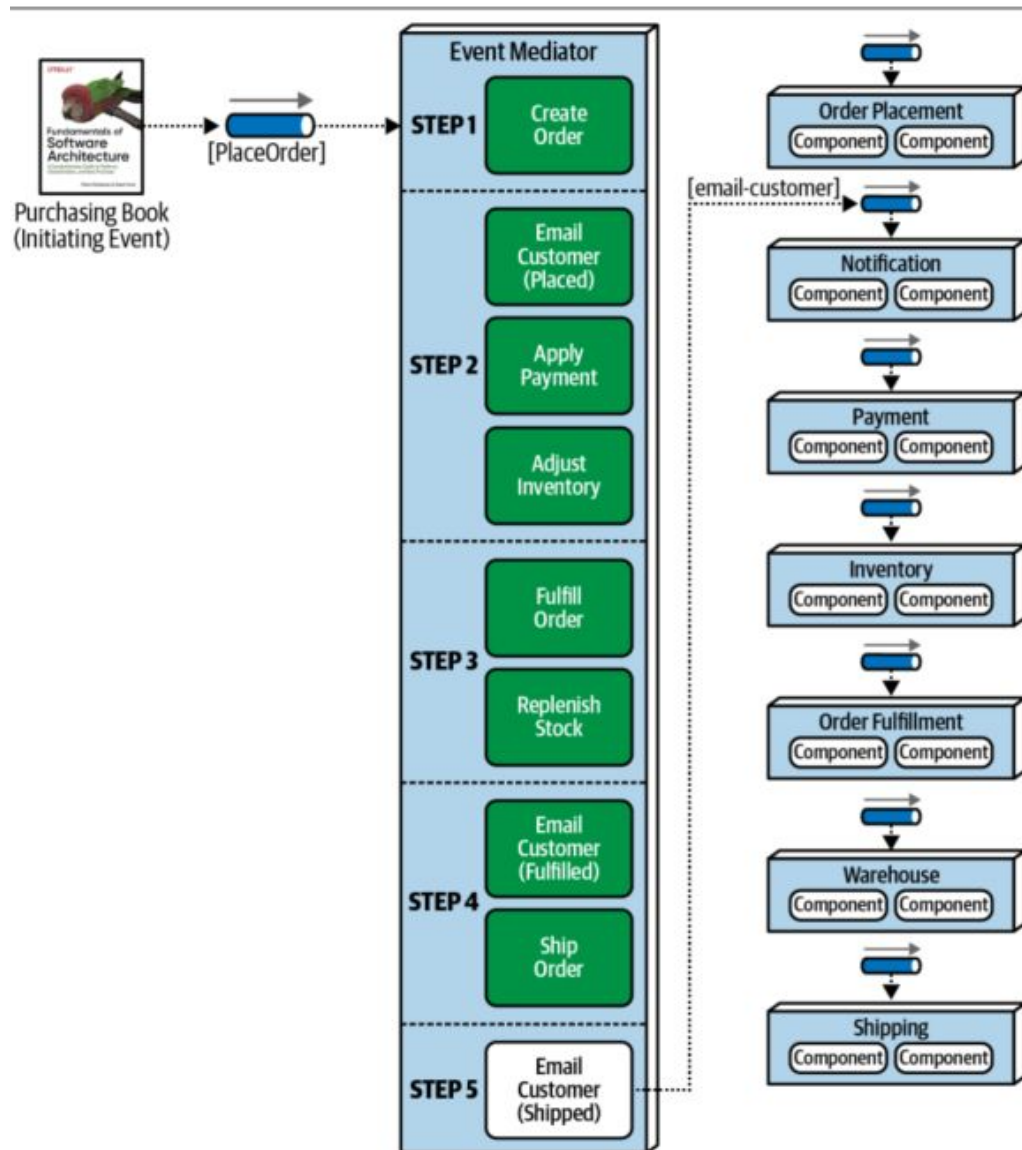
Event-Driven Architecture



Event-Driven Architecture



Event-Driven Architecture



Event-Driven Architecture

Broker Example

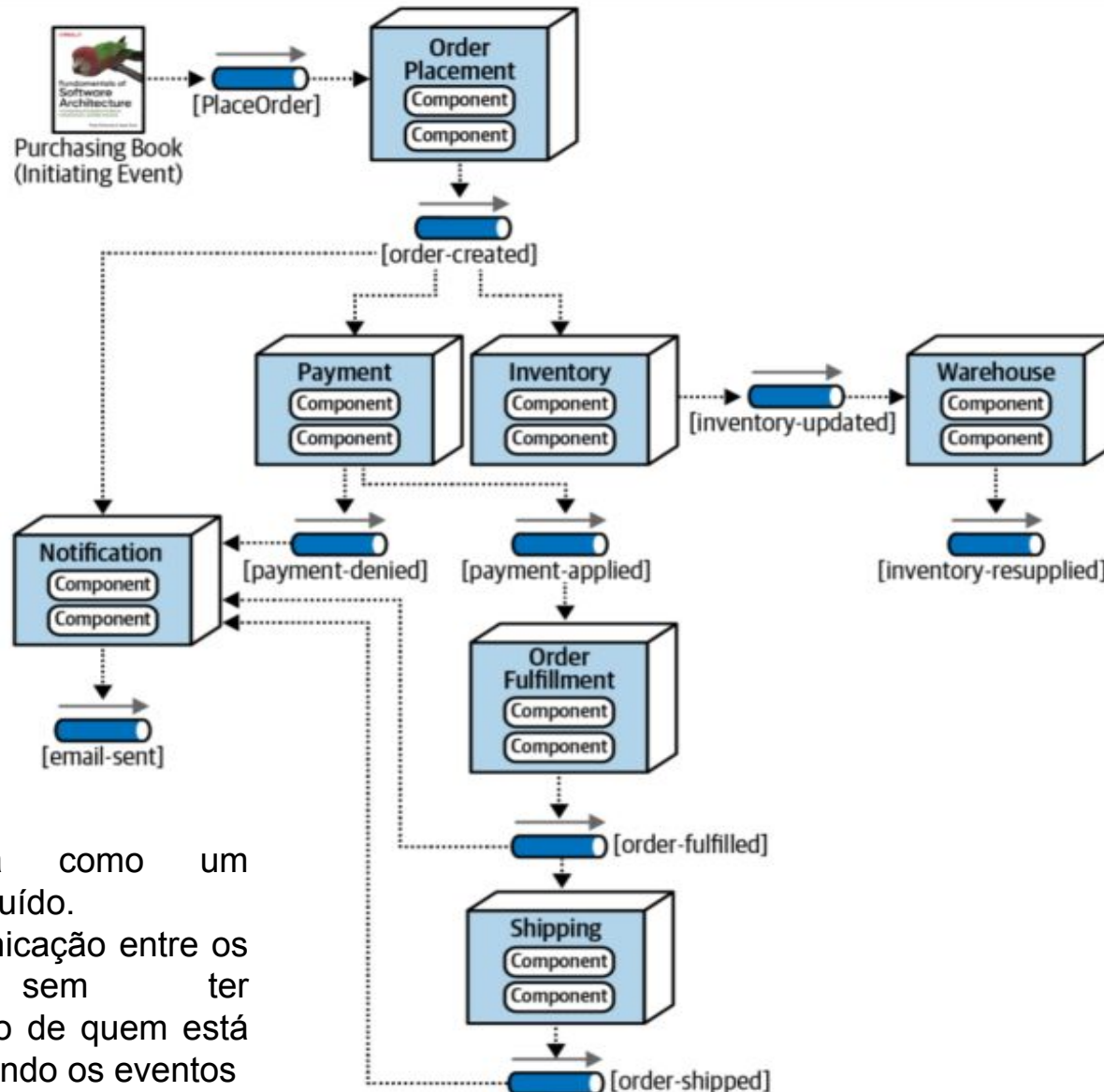


Figure 14-4. Example of the broker topology

O Broker atua como um intermediário distribuído. Ele facilita a comunicação entre os componentes sem ter conhecimento direto de quem está enviando ou recebendo os eventos

Event-Driven Architecture

- É perceptível que por ser uma comunicação assíncrona, a perda de dados ou requisições deve ser tratada de forma mais protetiva, evitando a perda de requisições ou dados durante o processo de comunicação dos módulos ou servidores.
- Por exemplo, a requisição pode ser
 - parcialmente executada no servidor;
 - haver uma falha na validação dos dados;
 - modelo do banco de dados rejeitar a transação;
 - dados serem perdidos no roteamento na internet; etc...

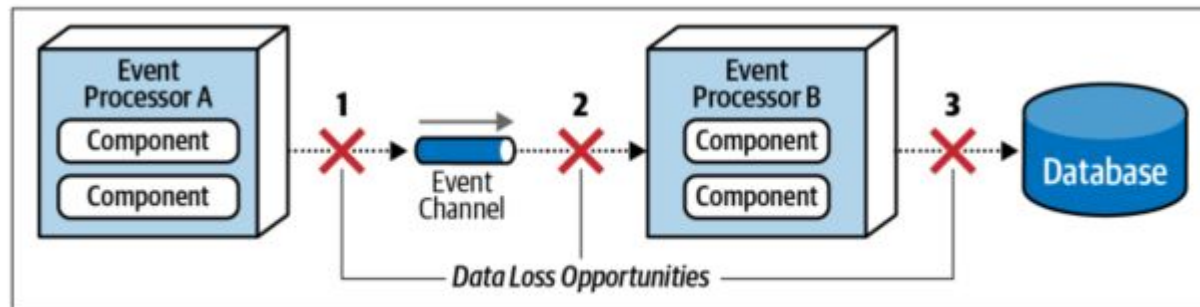


Figure 14-16. Where data loss can happen within an event-driven architecture

Event-Driven Architecture

- Cada evento que pode causar prejuízo aos dados ou transações possui técnicas específicas que podem ser agregadas para reduzir o risco.
- Por exemplo, usar filas persistentes (o evento somente é removido da fila após ser totalmente efetivado por todos os servidores).

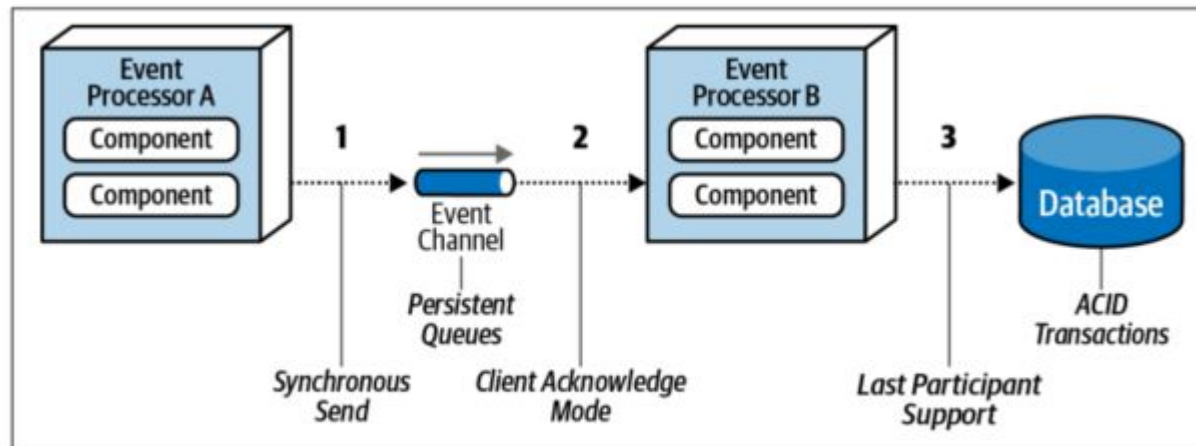


Figure 14-17. Preventing data loss within an event-driven architecture

Event-Driven Architecture

- Outra técnica que pode ser utilizada é o **broadcast** do evento para todos os módulos e servidores, onde cada um pode decidir o que fazer com aquele evento.
- Se não for “para mim” eu descarto o evento e volto a escutar a rede.
- Se houver mais de uma instância do mesmo servidor ou módulo executando pode haver uma validação no momento da efetivação da transação.
- Quem terminar primeiro registra a finalização, as propriedades do Banco de Dados garantem que não haja duplicação.
 - Infelizmente haverá trabalho “em vão”, pois um servidor processou a requisição e o outro também.

Event-Driven Architecture

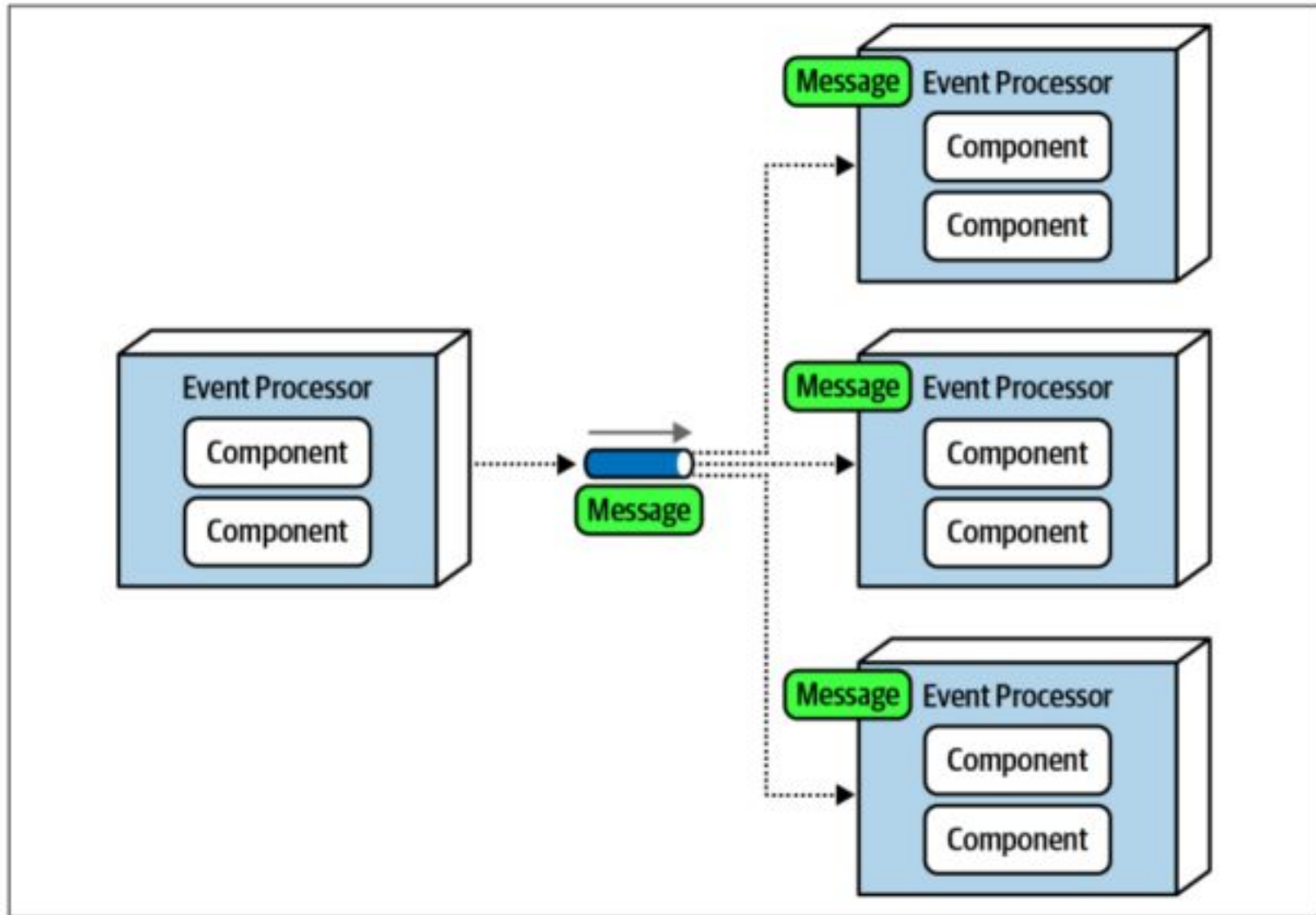


Figure 14-18. Broadcasting events to other event processors

Event-Driven Architecture

- Outra técnica que pode ser utilizada para prevenção de perda dos dados é a **mensagem de reconhecimento**.
- Parece reduzir os benefícios da abordagem distribuída?

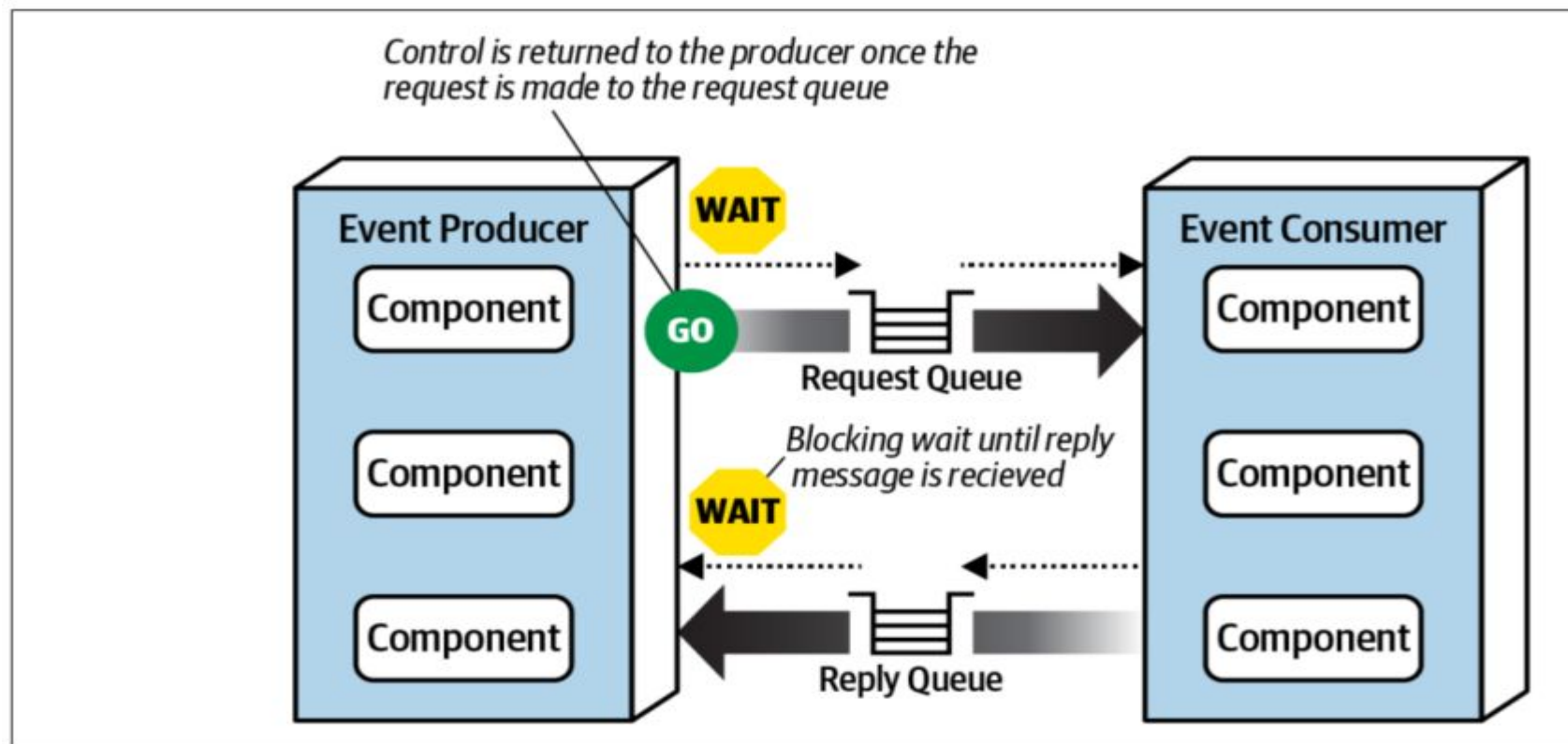


Figure 14-19. Request-reply message processing

Event-Driven Architecture

- Uma forma de implementar esta técnica é utilizando IDs na requisição e nas respostas, podendo associar que evento XY foi processado com sucesso.
 - CID = Correlation ID
 - A mensagem de resposta tem o seu próprio ID único (novo), e o CID transporta o ID do pedido original para efeitos de rastreamento

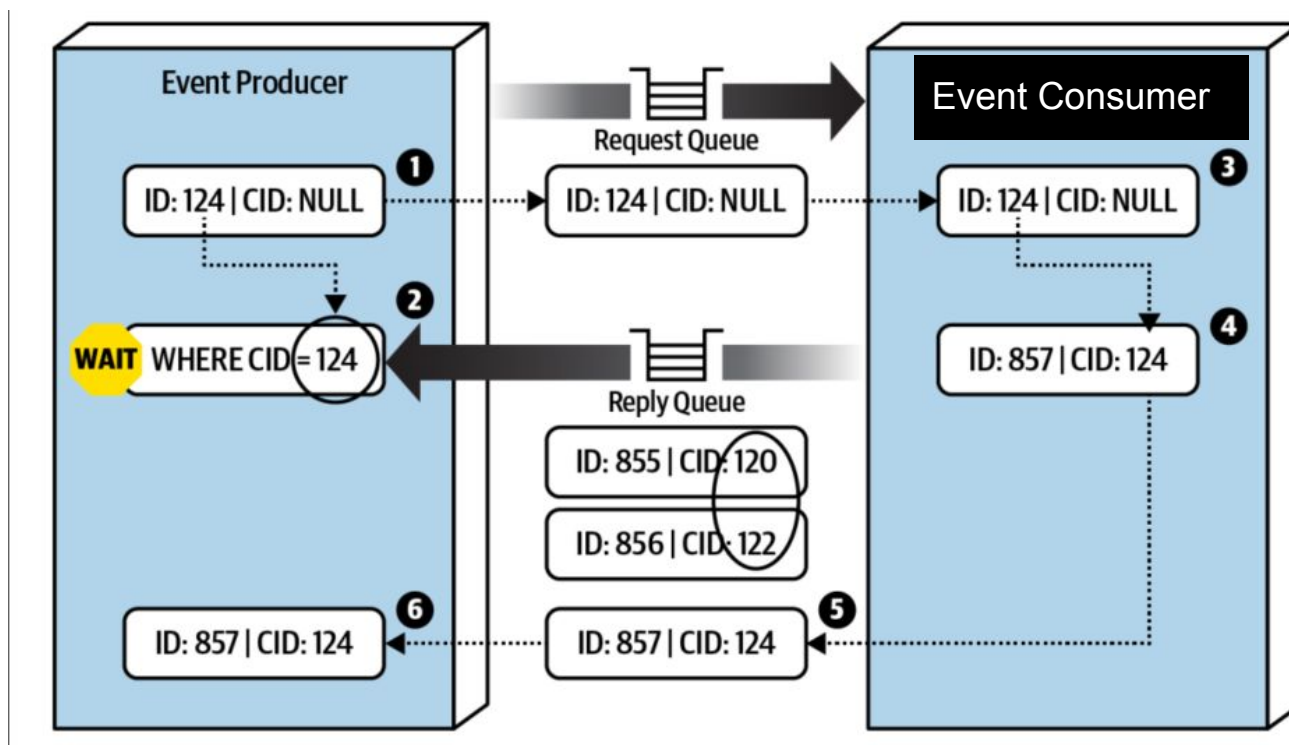


Figure 14-20. Request-reply message processing using a correlation ID

Event-Driven Architecture

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1 to many
Deployability	★ ★ ★
Elasticity	★ ★ ★
Evolutionary	★ ★ ★ ★ ★
Fault tolerance	★ ★ ★ ★ ★
Modularity	★ ★ ★ ★
Overall cost	★ ★ ★
Performance	★ ★ ★ ★ ★
Reliability	★ ★ ★
Scalability	★ ★ ★ ★ ★
Simplicity	★
Testability	★ ★

Event-Driven Architecture

- A quantidade de quanta na arquitetura baseada em eventos pode variar de 1 para muitos, dependendo principalmente da quantidade de bancos de dados, sendo que o serviço pode compartilhar o banco de dados.
- Mesmo usando principalmente requisições assíncronas, se o banco de dados é compartilhado, não temos uma unidade deployável independente (quanta = 1)
- Exemplo
 - Um evento envia uma requisição para outro processador para executar uma ordem de compra. O primeiro evento precisa esperar pelo id do pedido para continuar o processamento.
 - Se o segundo evento estiver indisponível, o primeiro não conseguirá executar pela dependência, tornando-os um quanta único (quanta =1) mesmo se comunicando por mensagens assíncronas.

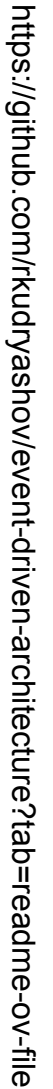
Event-Driven Architecture

- A arquitetura baseada em eventos apresenta 5 estrelas para desempenho, escalabilidade e tolerância a falhas, principais qualidades da arquitetura.
- Alto desempenho é alcançado pela comunicação assíncrona combinado com o processamento paralelizável.
- Alta escalabilidade é alcançada pelo balanceamento de eventos entre os computadores / serviços consumidores
 - Se a carga de processamento aumentar, é possível escalar novos consumidores.
- A tolerância a falhas é alcançada pelo alto desacoplamento dos componentes e os eventos assíncronos.

Event-Driven Architecture

- Por outro lado, é difícil realizar testes precisos, reduzindo as notas em simplicidade e testabilidade.
- Por possuir eventos assíncronos e não determinísticos, é difícil prever como um evento irá se comportar e ser integrado por outro evento.
- Por fim, a arquitetura baseada em eventos é altamente Evolucionária, é fácil adicionar funcionalidades adicionando novos processadores de eventos.
- Ademais, é preciso criar um novo formato do evento que será publicado em uma fila para ser lido por outro processador (consumidor)

<https://github.com/kudryashov/event-driven-architecture?tab=readme-ov-file>



Dúvidas?



www.shutterstock.com · 744867163